UNIVERSITE ABDELMALEK ESSAADI
ECOLE NATIONALE DES SCIENCES APPLIQUEE
D'AL HOCEIMA

Compte Rendu de TP4

Filière: Transformation digitale et Intelligence artificielle

Module : J2EE / Java EE / Jakarta EE

Titre :

Application CRUD – Gestion des Produits

Realise par :

- ✓ MAZGOUT Rim

Encadré par :

- ✓ Pr. CHERRADI Mohamed

Année universitaire : 2025/2026

Table des matières

Table des matières 1

Table des figures 3

1 Introduction 4

1.1 Contexte et Objectif 4

1.2 Technologies Utilisées 4

1.3 Fonctionnalités Développées 4

2 Architecture du Projet 5

2.1 Architecture MVC1 5

2.2 Architecture 3 Tiers 5

2.3 Structure des Dossiers du Projet 5

3 Configuration du Projet 7

3.1 Configuration Maven — pom.xml 7

3.2 Configuration — web.xml 7

4 Couche Model 9

4.1 Classe Produit.java 9

4.2 Classe Utilisateur.java 10

5 Couche DAO 11

5.1 Interface ProduitDAO.java 11

5.2 Implémentation ProduitImpl.java 11

6 Couche Métier (Service) 14

6.1 Interface ProduitMetier.java 14

6.2 Implémentation ProduitMetierImpl.java 14

7 Couche Web — Servlets 16

7.1 Présentation des Servlets 16

7.2 ListProduitServlet.java 16

7.3 AddProduitServlet.java 16

7.4 LoginServlet.java 17

8 Filtres — Sécurité et Gestion des Rôles 19

8.1 Principe des Filtres Java EE 19

8.2 Filtre AuthFilter.java 19
8.3 Filtre RoleFilter.java 20
8.4 Schéma du flux des filtres 20

9 Couche Vue — JSP **22**
9.1 Page login.jsp 22
9.2 Page index.jsp — Vue principale 22

10 Tests et Résultats **24**
10.1 Scénarios de Test 24
10.2 Captures d'écran de l'Application 24
10.2.1 Page de Connexion 25
10.2.2 Interface ADMIN 26
10.2.3 Interface USER 27
10.2.4 Pages d'Erreur 27

11 Problèmes Rencontrés et Solutions **29**
11.1 Erreur 404 au démarrage 29
11.2 Erreur de Doublons dans web.xml 29
11.3 USER pouvait accéder aux actions ADMIN 29

12 Conclusion **31**

A Comptes Utilisateurs de Test **32**

B URLs de l'Application **33**

Table des figures

3.1 Fichier pom.xml 7
3.2 Fichier web.xml 8

4.1 Fichier Produit.java 9
4.2 Fichier Utilisateur.java 10

5.1 Interface ProduitDAO.java 11
5.2 Implémentation ProduitImpl.java 12
5.3 Package com.gestion.dao 13

6.1 Interface ProduitMetier.java 14
6.2 Interface ProduitMetier.java 14
6.3 Implémentation ProduitMétierImpl.java 15

7.1 ListProduitServlet.java 16
7.2 AddProduitServlet.java 17
7.3 LoginServlet.java 18
7.4 Package com.gestion.web 18

8.1 Filtre d'authentification 19
8.2 Filtre de rôles 20
8.3 Package com.gestion.filter 21

9.1 Extrait de login.jsp 22
9.2 Extrait de index.jsp Vue Admin 23
9.3 Extrait de index.jsp Vue User 23

10.1 Formulaire de connexion 25
10.2 Message d'erreur pour identifiants incorrects 25
10.3 Tableau des produits avec boutons Modifier/Supprimer 26
10.4 Formulaire d'ajout d'un nouveau produit 26
10.5 Formulaire de modification d'un produit existant 26
10.6 Page d'accueil USER 27
10.7 Accès refusé pour un USER tentant une action ADMIN 27
10.8 Page introuvable 28

1. Introduction

1.1 Contexte et Objectif

Ce travail pratique a pour objectif de concevoir et développer une application web Java EE de gestion de produits, basée sur l'architecture **MVC1** (Model-View-Controller) combinée avec une organisation en **3 tiers** (Présentation, Métier, Données).

L'application permet de réaliser l'ensemble des opérations **CRUD** (Create, Read, Update, Delete) sur des produits, tout en intégrant un système d'authentification par filtres et une gestion des rôles utilisateurs.

1.2 Technologies Utilisées

TABLE 1.1 – Technologies et outils du projet

Technologie	Rôle	Version
Java EE	Langage principal côté serveur	Java 8
Servlet	Contrôleurs de l'architecture MVC1	API 4.0.1
JSP	Vues de l'application	API 2.3.3
JSTL	Balises JSP avancées	1.2
Apache Tomcat	Serveur d'application	9.0.115
Maven	Gestion des dépendances et build	3.x
Eclipse IDE	Environnement de développement	2023

1.3 Fonctionnalités Développées

- **Gestion des produits** : affichage, ajout, modification, suppression d'un produit (CRUD complet).
- **Authentification** : connexion/déconnexion via session HTTP et filtre de sécurité.
- **Gestion des rôles** :
  - ADMIN : accès complet (CRUD + ajout/modification/suppression).
  - USER : accès en lecture seule.
- **Gestion des erreurs** : pages personnalisées pour les erreurs 403, 404 et 500.

2. Architecture du Projet

2.1 Architecture MVC1

L'architecture **MVC1** (ou *Model 1*) utilisée dans ce projet repose sur le principe suivant :

— **Model** : classes Java représentant les données métier (Produit, Utilisateur).

— **View** : pages JSP affichant les données à l'utilisateur (index.jsp, login.jsp).

— **Controller** : Servlets recevant les requêtes HTTP, appellant la couche métier et redirigeant vers la vue.

2.2 Architecture 3 Tiers

TABLE 2.1 – Organisation en 3 tiers

Tier	Couche	Classes / Fichiers
Présentation	Vue Contrôleur	index.jsp, login.jsp *Servlet.java
Métier	Service	ProduitMetier, ProduitMetierImpl
Données	DAO	ProduitDAO, ProduitImpl

2.3 Structure des Dossiers du Projet

Arborescence du projet Maven

```
GestionProduits/
 src/
    main/
        java/
            com/gestion/
                model/        ← Produit.java, Utilisateur.java
                dao/          ← ProduitDAO, ProduitImpl,
                                UtilisateurDAO, UtilisateurImpl
                metier/       ← ProduitMetier, ProduitMetierImpl
                web/          ← *Servlet.java, LoginServlet,
                                LogoutServlet
                filter/       ← AuthFilter.java, RoleFilter.java
        webapp/
            WEB-INF/
```

```
                    web.xml
                index.jsp
                login.jsp
                error403.jsp
                error404.jsp
                error500.jsp
pom.xml
```

3. Configuration du Projet

3.1 Configuration Maven — pom.xml

fichier pom.xml de configuration Maven. Définit l'identité du projet (groupId, artifactId, packaging war), les dépendances nécessaires (javax.servlet-api, javax.servlet.jsp-api, jstl) avec leurs versions compatibles Tomcat 9, et les plugins de compilation et de génération du fichier WAR.

```xml

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.gestion</groupId>
    <artifactId>GestionProduits</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>
    <name>GestionProduits</name>

    <properties>
        <maven.compiler.source>1.8</maven.compiler.source>
        <maven.compiler.target>1.8</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>

        <!-- 1. Servlet API Tomcat 9 → javax -->
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>

        <!-- 2. JSP API Tomcat 9 → javax -->
        <dependency>
            <groupId>javax.servlet.jsp</groupId>
            <artifactId>javax.servlet.jsp-api</artifactId>
            <version>2.3.3</version>
            <scope>provided</scope>
        </dependency>

        <!-- 3. JSTL Tomcat 9 → javax (une seule dépendance suffit) -->
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>jstl</artifactId>
            <version>1.2</version>
        </dependency>

    </dependencies>
```

FIGURE 3.1 – Fichier pom.xml

3.2 Configuration — web.xml

fichier web.xml de configuration central de l'application web. Déclare et mappe toutes les Servlets avec leurs URLs, enregistre les deux filtres avec leurs URL-patterns, configure les pages d'erreur personnalisées (403, 404, 500) et définit la page d'accueil par défaut.

```
10    <servlet>
11        <servlet-name>LoginServlet</servlet-name>
12        <servlet-class>com.gestion.web.LoginServlet</servlet-class>
13    </servlet>
14    <servlet-mapping>
15        <servlet-name>LoginServlet</servlet-name>
16        <url-pattern>/login</url-pattern>
17    </servlet-mapping>
18
19    <servlet>
20        <servlet-name>LogoutServlet</servlet-name>
21        <servlet-class>com.gestion.web.LogoutServlet</servlet-class>
22    </servlet>
23    <servlet-mapping>
24        <servlet-name>LogoutServlet</servlet-name>
25        <url-pattern>/logout</url-pattern>
26    </servlet-mapping>
27
28    <servlet>
29        <servlet-name>ListProduitServlet</servlet-name>
30        <servlet-class>com.gestion.web.ListProduitServlet</servlet-class>
31    </servlet>
32    <servlet-mapping>
33        <servlet-name>ListProduitServlet</servlet-name>
34        <url-pattern>/produits</url-pattern>
35    </servlet-mapping>
36
37    <servlet>
38        <servlet-name>AddProduitServlet</servlet-name>
39        <servlet-class>com.gestion.web.AddProduitServlet</servlet-class>
40    </servlet>
41    <servlet-mapping>
42        <servlet-name>AddProduitServlet</servlet-name>
43        <url-pattern>/addProduit</url-pattern>
44    </servlet-mapping>
45
46    <servlet>
47        <servlet-name>DeleteProduitServlet</servlet-name>
48        <servlet-class>com.gestion.web.DeleteProduitServlet</servlet-class>
49    </servlet>
50    <servlet-mapping>
51        <servlet-name>DeleteProduitServlet</servlet-name>
52        <url-pattern>/deleteProduit</url-pattern>
```

```
54
55    <servlet>
56        <servlet-name>EditProduitServlet</servlet-name>
57        <servlet-class>com.gestion.web.EditProduitServlet</servlet-class>
58    </servlet>
59    <servlet-mapping>
60        <servlet-name>EditProduitServlet</servlet-name>
61        <url-pattern>/editProduit</url-pattern>
62    </servlet-mapping>
63
64    <servlet>
65        <servlet-name>UpdateProduitServlet</servlet-name>
66        <servlet-class>com.gestion.web.UpdateProduitServlet</servlet-class>
67    </servlet>
68    <servlet-mapping>
69        <servlet-name>UpdateProduitServlet</servlet-name>
70        <url-pattern>/updateProduit</url-pattern>
71    </servlet-mapping>
72
73
74
75    <filter>
76        <filter-name>AuthFilter</filter-name>
77        <filter-class>com.gestion.filter.AuthFilter</filter-class>
78    </filter>
79    <filter-mapping>
80        <filter-name>AuthFilter</filter-name>
81        <url-pattern>/*</url-pattern>
82    </filter-mapping>
83
84    <filter>
85        <filter-name>RoleFilter</filter-name>
86        <filter-class>com.gestion.filter.RoleFilter</filter-class>
87    </filter>
88    <filter-mapping>
89        <filter-name>RoleFilter</filter-name>
90        <url-pattern>/addProduit</url-pattern>
91    </filter-mapping>
92    <filter-mapping>
93        <filter-name>RoleFilter</filter-name>
94        <url-pattern>/deleteProduit</url-pattern>
95    </filter-mapping>
96    <filter-mapping>
97        <filter-name>RoleFilter</filter-name>
98        <url-pattern>/editProduit</url-pattern>
99    </filter-mapping>
100   <filter-mapping>
101       <filter-name>RoleFilter</filter-name>
102       <url-pattern>/updateProduit</url-pattern>
103   </filter-mapping>
```

```
106
107   <error-page>
108       <error-code>403</error-code>
109       <location>/error403.jsp</location>
110   </error-page>
111
112   <error-page>
113       <error-code>404</error-code>
114       <location>/error404.jsp</location>
115   </error-page>
116
117   <error-page>
118       <error-code>500</error-code>
119       <location>/error500.jsp</location>
120   </error-page>
121
122   <error-page>
123       <exception-type>java.lang.Exception</exception-type>
124       <location>/error500.jsp</location>
125   </error-page>
126
```

FIGURE 3.2 – Fichier web.xml

4. Couche Model

4.1 Classe Produit.java

La classe Produit représente l'entité Produit avec ses 4 attributs (idProduit, nom, description, prix), ses constructeurs, getters/setters et toString(). C'est le modèle de données central de l'application.

```java
package com.gestion.model;

public class Produit {

    // ============================================
    //              ATTRIBUTS
    // ============================================
    private int idProduit;
    private String nom;
    private String description;
    private double prix;

    // ============================================
    //          CONSTRUCTEUR VIDE
    // ============================================
    public Produit() {
    }

    // ============================================
    //         CONSTRUCTEUR AVEC PARAMÈTRES
    // ============================================
    public Produit(int idProduit, String nom, String description, double prix) {
        this.idProduit   = idProduit;
        this.nom         = nom;
        this.description = description;
        this.prix        = prix;
    }

    // ============================================
    //              GETTERS & SETTERS
    // ============================================

    public int getIdProduit() {
        return idProduit;
    }

    public void setIdProduit(int idProduit) {
        this.idProduit = idProduit;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
```

```java
    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrix() {
        return prix;
    }

    public void setPrix(double prix) {
        this.prix = prix;
    }

    // ============================================
    //              toString()
    // ============================================
    @Override
    public String toString() {
        return "Produit{" +
                "idProduit=" + idProduit +
                ", nom='"         + nom         + '\'' +
                ", description='" + description + '\'' +
                ", prix="         + prix        +
                '}';
    }
}
```

FIGURE 4.1 – Fichier Produit.java

4.2 Classe Utilisateur.java

La classe Utilisateur représente un utilisateur du système avec ses attributs (idUtilisateur, login, motDePasse, role). Elle permet de gérer l'authentification et les droits d'accès.

```java
package com.gestion.model;

public class Utilisateur {

    private int idUtilisateur;
    private String login;
    private String motDePasse;
    private String role;

    public Utilisateur() {}

    public Utilisateur(int idUtilisateur, String login,
                       String motDePasse, String role) {
        this.idUtilisateur = idUtilisateur;
        this.login         = login;
        this.motDePasse    = motDePasse;
        this.role          = role;
    }

    public int getIdUtilisateur()          { return idUtilisateur; }
    public void setIdUtilisateur(int id)   { this.idUtilisateur = id; }

    public String getLogin()               { return login; }
    public void setLogin(String login)     { this.login = login; }

    public String getMotDePasse()          { return motDePasse; }
    public void setMotDePasse(String mdp)  { this.motDePasse = mdp; }

    public String getRole()                { return role; }
    public void setRole(String role)       { this.role = role; }
```

FIGURE 4.2 – Fichier Utilisateur.java

5. Couche DAO

5.1 Interface ProduitDAO.java

L'interface ProduitDAO définit le contrat des opérations CRUD sur les produits. Elle déclare les 5 méthodes : addProduit, getAllProduits, getProduitById, updateProduit, deleteProduit. Aucune logique, uniquement les signatures.

```
web.xml   index.jsp   GestionProdu...   Produit.java   ProduitDAO.java   UpdateProdui
1  package com.gestion.dao;
2
3  import com.gestion.model.Produit;
4  import java.util.List;
5
6  public interface ProduitDAO {
7
8      // Ajouter un nouveau produit
9      void addProduit(Produit produit);
10
11     // Récupérer tous les produits
12     List<Produit> getAllProduits();
13
14     // Récupérer un produit par son ID
15     Produit getProduitById(int id);
16
17     // Mettre à jour un produit existant
18     void updateProduit(Produit produit);
19
20     // Supprimer un produit par son ID
21     void deleteProduit(int id);
```

FIGURE 5.1 – Interface ProduitDAO.java

5.2 Implémentation ProduitImpl.java

ProduitImpl implémente ProduitDAO avec un stockage en mémoire via une ArrayList statique simulant une base de données.

```java
package com.gestion.dao;

import com.gestion.model.Produit;
import java.util.ArrayList;
import java.util.List;

public class ProduitImpl implements ProduitDAO {

    // ==========================================
    //   STOCKAGE EN MÉMOIRE (ArrayList statique)
    // ==========================================
    private static List<Produit> listeProduits = new ArrayList<>();
    private static int compteurId = 1;

    // Données de test préchargées au démarrage
    static {
        listeProduits.add(new Produit(compteurId++, "Laptop Dell",
                "Ordinateur portable 15 pouces", 8500.00));
        listeProduits.add(new Produit(compteurId++, "Souris Logitech",
                "Souris sans fil ergonomique", 250.00));
        listeProduits.add(new Produit(compteurId++, "Clavier Mécanique",
                "Clavier RGB switches bleus", 450.00));
    }

    // ==========================================
    //   CREATE — Ajouter un produit
    // ==========================================
    @Override
    public void addProduit(Produit produit) {
        produit.setIdProduit(compteurId++);
        listeProduits.add(produit);
    }

    // ==========================================
    //   READ ALL — Récupérer tous les produits
    // ==========================================
    @Override
    public List<Produit> getAllProduits() {
        return listeProduits;
    }


    // ==========================================
    //   READ ONE — Récupérer un produit par ID
    // ==========================================
    @Override
    public Produit getProduitById(int id) {
        for (Produit p : listeProduits) {
            if (p.getIdProduit() == id) {
                return p;
            }
        }
        return null;
    }

    // ==========================================
    //   UPDATE — Modifier un produit existant
    // ==========================================
    @Override
    public void updateProduit(Produit produit) {
        for (int i = 0; i < listeProduits.size(); i++) {
            if (listeProduits.get(i).getIdProduit() == produit.getIdProduit()) {
                listeProduits.set(i, produit);
                break;
            }
        }
    }

    // ==========================================
    //   DELETE — Supprimer un produit par ID
    // ==========================================
    @Override
    public void deleteProduit(int id) {
        listeProduits.removeIf(p -> p.getIdProduit() == id);
    }
```

FIGURE 5.2 – Implémentation ProduitImpl.java

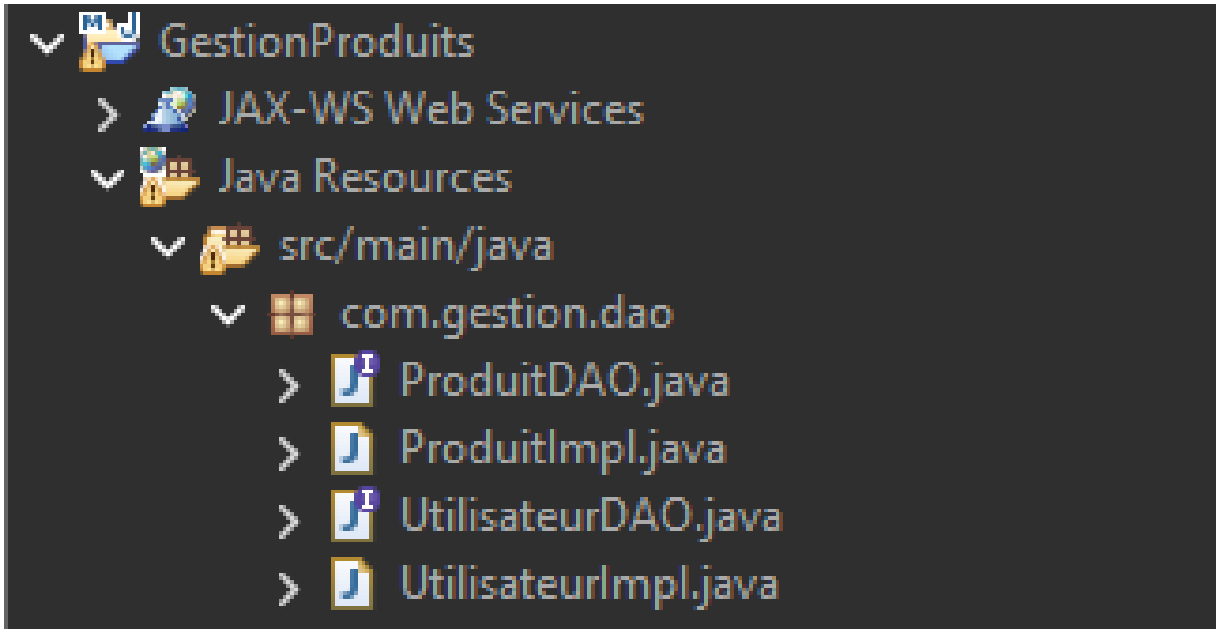

FIGURE 5.3 – Package com.gestion.dao

6. Couche Métier (Service)

6.1 Interface ProduitMetier.java

Interface de la couche service. Elle définit les mêmes 5 opérations CRUD que Produit-DAO mais constitue une couche d'abstraction entre les Servlets et le DAO, permettant d'ajouter de la logique métier sans toucher aux contrôleurs.

```java
package com.gestion.metier;

import com.gestion.model.Produit;
import java.util.List;

public interface ProduitMetier {
    void addProduit(Produit produit);
    List<Produit> getAllProduits();
    Produit getProduitById(int id);
    void updateProduit(Produit produit);
    void deleteProduit(int id);
```

FIGURE 6.1 – Interface ProduitMetier.java

6.2 Implémentation ProduitMetierImpl.java

La couche service délègue entièrement les opérations à la couche DAO. Elle constitue une **façade** entre les Servlets et le DAO.

```java
package com.gestion.metier;

import com.gestion.model.Produit;
import java.util.List;

public interface ProduitMetier {
    void addProduit(Produit produit);
    List<Produit> getAllProduits();
    Produit getProduitById(int id);
    void updateProduit(Produit produit);
    void deleteProduit(int id);
```

FIGURE 6.2 – Interface ProduitMetier.java

```java
package com.gestion.metier;

import com.gestion.dao.ProduitDAO;
import com.gestion.dao.ProduitImpl;
import com.gestion.model.Produit;
import java.util.List;

public class ProduitMetierImpl implements ProduitMetier {

    private ProduitDAO produitDAO = new ProduitImpl();

    @Override
    public void addProduit(Produit produit) {
        produitDAO.addProduit(produit);
    }

    @Override
    public List<Produit> getAllProduits() {
        return produitDAO.getAllProduits();
    }

    @Override
    public Produit getProduitById(int id) {
        return produitDAO.getProduitById(id);
    }

    @Override
    public void updateProduit(Produit produit) {
        produitDAO.updateProduit(produit);
    }

    @Override
    public void deleteProduit(int id) {
        produitDAO.deleteProduit(id);
    }
}
```

FIGURE 6.3 – Implémentation ProduitMétierImpl.java

7. Couche Web — Servlets

7.1 Présentation des Servlets

TABLE 7.1 – Servlets et leurs responsabilités

Servlet	URL	Rôle
ListProduitServlet	/produits	Affiche la liste (GET)
AddProduitServlet	/addProduit	Ajoute un produit (POST)
DeleteProduitServlet	/deleteProduit	Supprime un produit (GET)
EditProduitServlet	/editProduit	Charge le formulaire de modification (GET)
UpdateProduitServlet	/updateProduit	Enregistre la modification (POST)
LoginServlet	/login	Gère la connexion (GET/POST)
LogoutServlet	/logout	Déconnecte l'utilisateur (GET)

7.2 ListProduitServlet.java

Contrôleur GET mappé sur /produits. Récupère la liste complète des produits via la couche métier, la stocke dans la requête et redirige vers index.jsp pour l'affichage.

```java
1  package com.gestion.web;
2
3  import com.gestion.metier.ProduitMetier;
4  import com.gestion.metier.ProduitMetierImpl;
5  import com.gestion.model.Produit;
6  import javax.servlet.ServletException;
7  import javax.servlet.http.HttpServlet;
8  import javax.servlet.http.HttpServletRequest;
9  import javax.servlet.http.HttpServletResponse;
10
11 import java.io.IOException;
12 import java.util.List;
13
14 public class ListProduitServlet extends HttpServlet {
15
16     private ProduitMetier produitMetier = new ProduitMetierImpl();
17
18     @Override
19     protected void doGet(HttpServletRequest request, HttpServletResponse response)
20             throws ServletException, IOException {
21
22         List<Produit> listeProduits = produitMetier.getAllProduits();
23         request.setAttribute("listeProduits", listeProduits);
24         request.getRequestDispatcher("/index.jsp").forward(request, response);
25     }
26 }
```

FIGURE 7.1 – ListProduitServlet.java

7.3 AddProduitServlet.java

Contrôleur POST mappé sur /addProduit. Récupère les données du formulaire d'ajout (nom, description, prix), crée un objet Produit et l'ajoute via la couche métier, puis redirige

vers la liste.

```java
package com.gestion.web;

import com.gestion.metier.ProduitMetier;
import com.gestion.metier.ProduitMetierImpl;
import com.gestion.model.Produit;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

import java.io.IOException;

public class AddProduitServlet extends HttpServlet {

    private ProduitMetier produitMetier = new ProduitMetierImpl();

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        String nom         = request.getParameter("nom");
        String description = request.getParameter("description");
        double prix        = Double.parseDouble(request.getParameter("prix"));

        Produit produit = new Produit();
        produit.setNom(nom);
        produit.setDescription(description);
        produit.setPrix(prix);

        produitMetier.addProduit(produit);

        response.sendRedirect(request.getContextPath() + "/produits");
    }
}
```

FIGURE 7.2 – AddProduitServlet.java

7.4 LoginServlet.java

Contrôleur GET/POST mappé sur /login. En GET, affiche la page de connexion. En POST, vérifie les identifiants saisis, crée une session HTTP avec les informations de l'utilisateur et son rôle si la connexion réussit, ou renvoie un message d'erreur sinon.

```java
public class LoginServlet extends HttpServlet {

    private UtilisateurDAO utilisateurDAO = new UtilisateurImpl();

    // Afficher la page de login
    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {
        request.getRequestDispatcher("/login.jsp")
                .forward(request, response);
    }

    // Traiter le formulaire de login
    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");

        String login      = request.getParameter("login");
        String motDePasse = request.getParameter("motDePasse");

        Utilisateur utilisateur =
            utilisateurDAO.findByLoginAndPassword(login, motDePasse);

        if (utilisateur != null) {
            // Créer la session et stocker l'utilisateur
            HttpSession session = request.getSession();
            session.setAttribute("utilisateurConnecte", utilisateur);
            session.setAttribute("role", utilisateur.getRole());

            // Rediriger vers la liste des produits
            response.sendRedirect(
                request.getContextPath() + "/produits");
        } else {
            // Mauvais identifiants → retour login avec message
            request.setAttribute("erreurLogin",
                "Login ou mot de passe incorrect !");
            request.getRequestDispatcher("/login.jsp")
                    .forward(request, response);
        }
    }
```

FIGURE 7.3 – LoginServlet.java

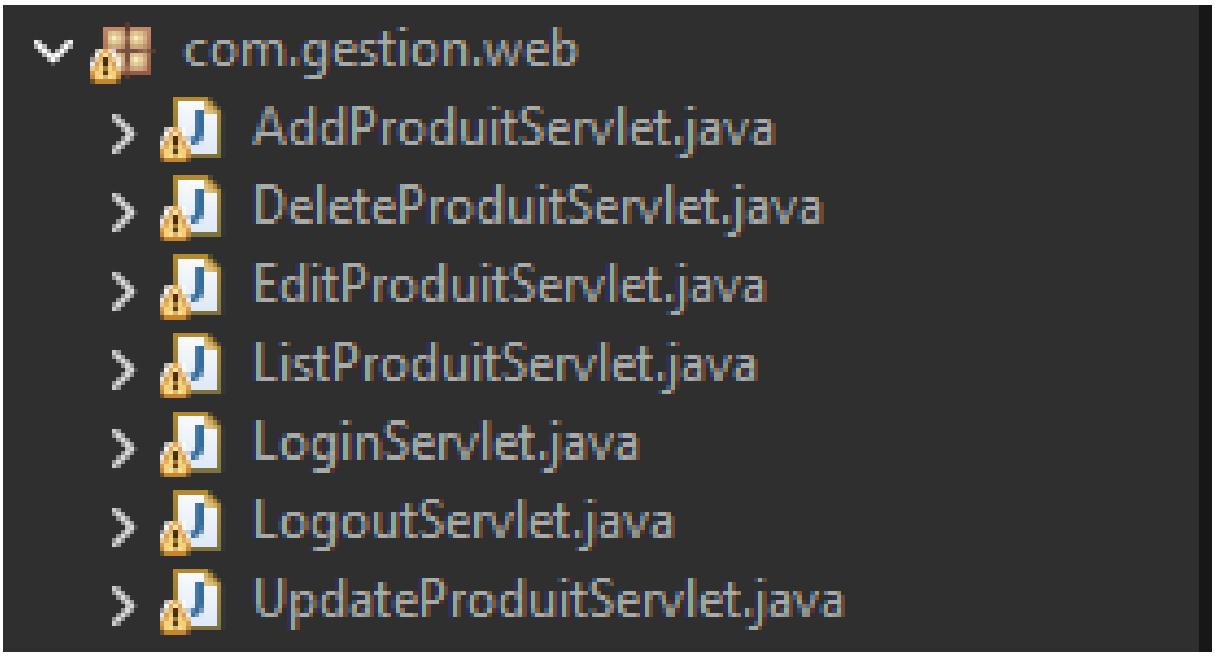

FIGURE 7.4 – Package com.gestion.web

8. Filtres — Sécurité et Gestion des Rôles

8.1 Principe des Filtres Java EE

Un **filtre** (`Filter`) est un composant Java EE qui intercepte les requêtes HTTP *avant* qu'elles atteignent le Servlet cible. Dans ce projet, deux filtres sont utilisés :

— **AuthFilter** : vérifie que l'utilisateur est connecté.

— **RoleFilter** : vérifie que l'utilisateur a le bon rôle (`ADMIN`) pour les opérations de modification.

8.2 Filtre AuthFilter.java

Filtre appliqué sur toutes les URLs (/*). Intercepte chaque requête HTTP et vérifie si l'utilisateur possède une session active. Si non, il est redirigé vers la page de login. Les URLs publiques (login, logout, ressources statiques) sont exemptées.

```java
 8  import javax.servlet.ServletResponse;
 9  import javax.servlet.http.HttpServletRequest;
10  import javax.servlet.http.HttpServletResponse;
11  import javax.servlet.http.HttpSession;
12  import java.io.IOException;
13
14  public class AuthFilter implements Filter {
15
16      @Override
17      public void init(FilterConfig filterConfig) throws ServletException {}
18
19      @Override
20      public void doFilter(ServletRequest request, ServletResponse response,
21                           FilterChain chain)
22              throws IOException, ServletException {
23
24          HttpServletRequest  req  = (HttpServletRequest)  request;
25          HttpServletResponse resp = (HttpServletResponse) response;
26
27          HttpSession session = req.getSession(false);
28
29          boolean estConnecte = (session != null &&
30                                 session.getAttribute("utilisateurConnecte") != null);
31
32          String uri = req.getRequestURI();
33
34          // laisser passer : login, logout, ressources statiques
35          boolean estPageLibre = uri.contains("/login")
36                               || uri.contains("/logout")
37                               || uri.contains(".css")
38                               || uri.contains(".js")
39                               || uri.contains(".png");
40
41          if (estConnecte || estPageLibre) {
42              // Utilisateur connecté ou page libre → on laisse passer
43              chain.doFilter(request, response);
44          } else {
45              // Non connecté → redirection vers login
46              resp.sendRedirect(req.getContextPath() + "/login");
47          }
48      }
49
50      @Override
51      public void destroy() {}
```

FIGURE 8.1 – Filtre d'authentification

8.3 Filtre RoleFilter.java

Filtre appliqué uniquement sur les URLs d'écriture (/addProduit, /deleteProduit, /editProduit, /updateProduit). Vérifie que l'utilisateur connecté possède le rôle ADMIN. Si le rôle est USER, il est redirigé vers la page d'erreur 403.

```java
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;
import java.io.IOException;

public class RoleFilter implements Filter {

    @Override
    public void init(FilterConfig filterConfig) throws ServletException {}

    @Override
    public void doFilter(ServletRequest request, ServletResponse response,
                         FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest  req  = (HttpServletRequest)  request;
        HttpServletResponse resp = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);
        String role = (session != null) ?
                      (String) session.getAttribute("role") : null;

        String uri = req.getRequestURI();

        // Actions réservées à l'ADMIN uniquement
        boolean estActionAdmin = uri.contains("/addProduit")
                               || uri.contains("/deleteProduit")
                               || uri.contains("/editProduit")
                               || uri.contains("/updateProduit");

        if (estActionAdmin && !"ADMIN".equals(role)) {
            // Pas le bon rôle → page 403
            resp.sendRedirect(req.getContextPath() + "/error403.jsp");
        } else {
            chain.doFilter(request, response);
        }
    }

    @Override
    public void destroy() {}
}
```

FIGURE 8.2 – Filtre de rôles

8.4 Schéma du flux des filtres

Flux d'une requête avec les filtres
Requête HTTP −→ AuthFilter −→ RoleFilter −→ Servlet −→ Vue JSP

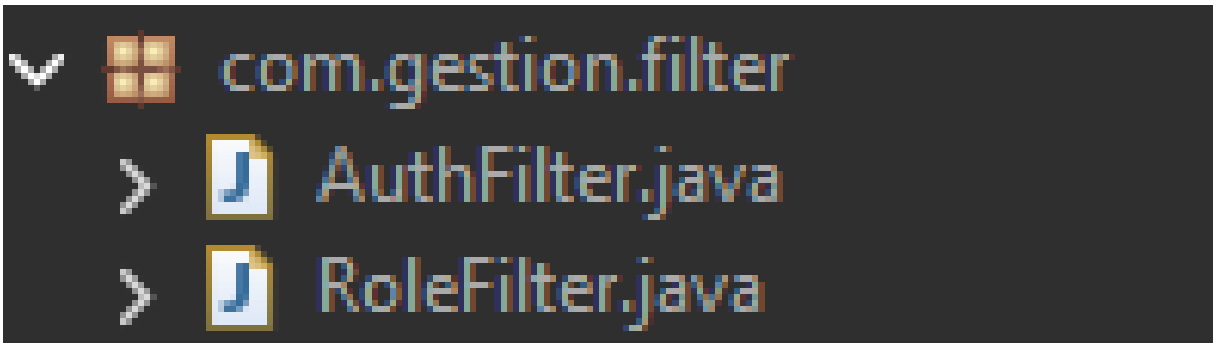

FIGURE 8.3 – Package com.gestion.filter

9. Couche Vue — JSP

9.1 Page login.jsp

La page de connexion présente un formulaire d'authentification. Elle affiche un message d'erreur si les identifiants sont incorrects.

```
75  <div class="login-box">
76      <h2>🔐 Connexion</h2>
77
78      <c:if test="${not empty erreurLogin}">
79          <div class="erreur">${erreurLogin}</div>
80      </c:if>
81
82      <form action="${pageContext.request.contextPath}/login" method="post">
83          <div class="form-group">
84              <label>Login</label>
85              <input type="text" name="login"
86                     placeholder="Entrez votre login" required />
87          </div>
88          <div class="form-group">
89              <label>Mot de passe</label>
90              <input type="password" name="motDePasse"
91                     placeholder="Entrez votre mot de passe" required />
92          </div>
93          <button type="submit" class="btn-login">Se connecter</button>
94      </form>
95
96      <div class="comptes">
97          👤 admin / admin123  |  👤 user / user123
98      </div>
99  </div>
100 </body>
101 </html>
```

FIGURE 9.1 – Extrait de login.jsp

9.2 Page index.jsp — Vue principale

La page index.jsp constitue la vue principale. Elle affiche :

— La barre de navigation avec le rôle de l'utilisateur connecté.

— Le tableau des produits (pour tous les utilisateurs).

— Les boutons Modifier/Supprimer et le formulaire d'ajout **uniquement pour l'ADMIN**.

— Un message d'information en lecture seule pour le rôle USER.

```
203 <c:if test="${empty produit and role == 'ADMIN'}">
204     <div class="form-container">
205         <h2>+ Ajouter un produit</h2>
206         <form action="${pageContext.request.contextPath}/addProduit"
207               method="post">
208             <div class="form-group">
209                 <label>Nom</label>
210                 <input type="text" name="nom"
211                        placeholder="Nom du produit" required />
212             </div>
213             <div class="form-group">
214                 <label>Description</label>
215                 <textarea name="description"
216                           placeholder="Description du produit"></textarea>
217             </div>
218             <div class="form-group">
219                 <label>Prix (MAD)</label>
220                 <input type="number" step="0.01" name="prix"
221                        placeholder="0.00" required />
222             </div>
223             <button type="submit" class="btn btn-success">
224                 + Ajouter
225             </button>
226         </form>
227     </div>
228 </c:if>
```

FIGURE 9.2 – Extrait de index.jsp Vue Admin

```
154 <!-- ============================================================ -->
155 <!--   MESSAGE pour USER : lecture seule                          -->
156 <!-- ============================================================ -->
157 <c:if test="${role == 'USER'}">
158     <div class="alert-info">
159         ℹ Vous êtes connecté en mode <strong>lecture seule</strong>.
160         Les opérations d'ajout, modification et suppression
161         sont réservées aux administrateurs.
162     </div>
163 </c:if>
164
```

FIGURE 9.3 – Extrait de index.jsp Vue User

10. Tests et Résultats

10.1 Scénarios de Test

TABLE 10.1 – Résultats des tests fonctionnels

Scénario	Action	Résultat
Accès sans connexion	Accéder à /produits directement	Redirigé vers /login
Connexion ADMIN	Login : admin / admin123	Accès complet, CRUD visible
Connexion USER	Login : user / user123	Liste visible, boutons cachés
Mauvais identifiants	Login incorrect	Message d'erreur affiché
Ajout produit (ADMIN)	Remplir formulaire et valider	Produit ajouté, redirection liste
Modification (ADMIN)	Cliquer Modifier, changer les champs	Produit mis à jour
Suppression (ADMIN)	Cliquer Supprimer avec confirmation	Produit supprimé de la liste
Accès interdit (USER)	USER tente /addProduit en URL	Page d'erreur 403 affichée
URL inexistante	Accéder à une URL inconnue	Page d'erreur 404 affichée
Déconnexion	Cliquer sur Déconnexion	Session invalidée, retour login

10.2 Captures d'écran de l'Application

10.2.1 Page de Connexion

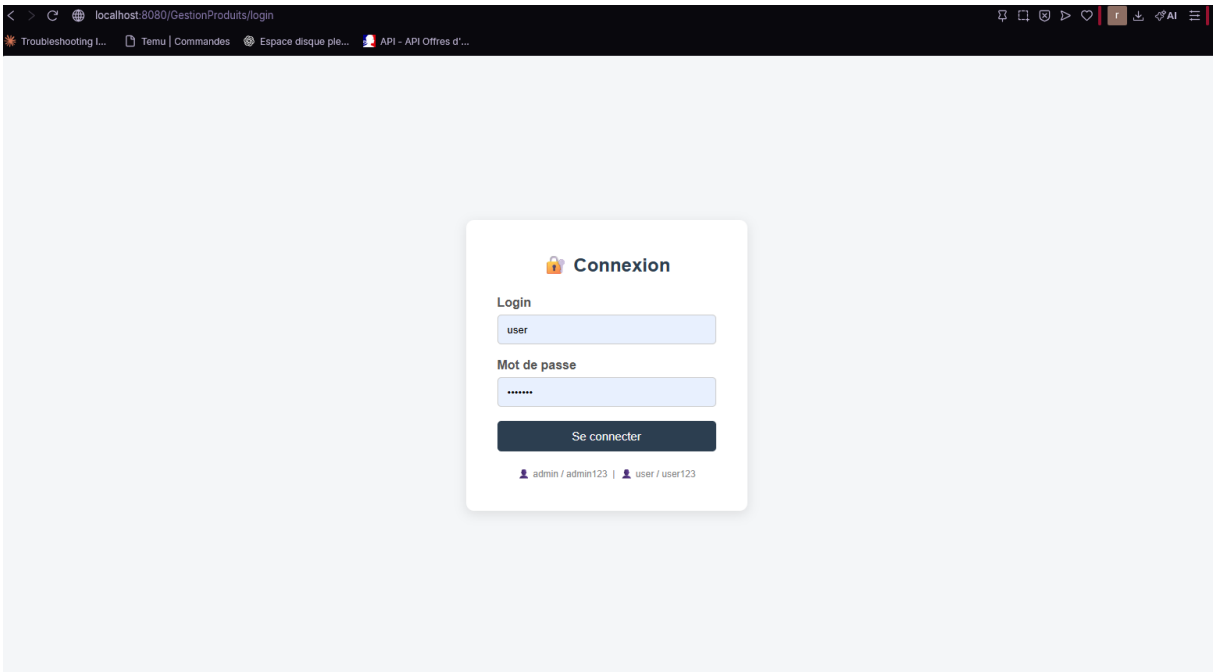

FIGURE 10.1 – Formulaire de connexion

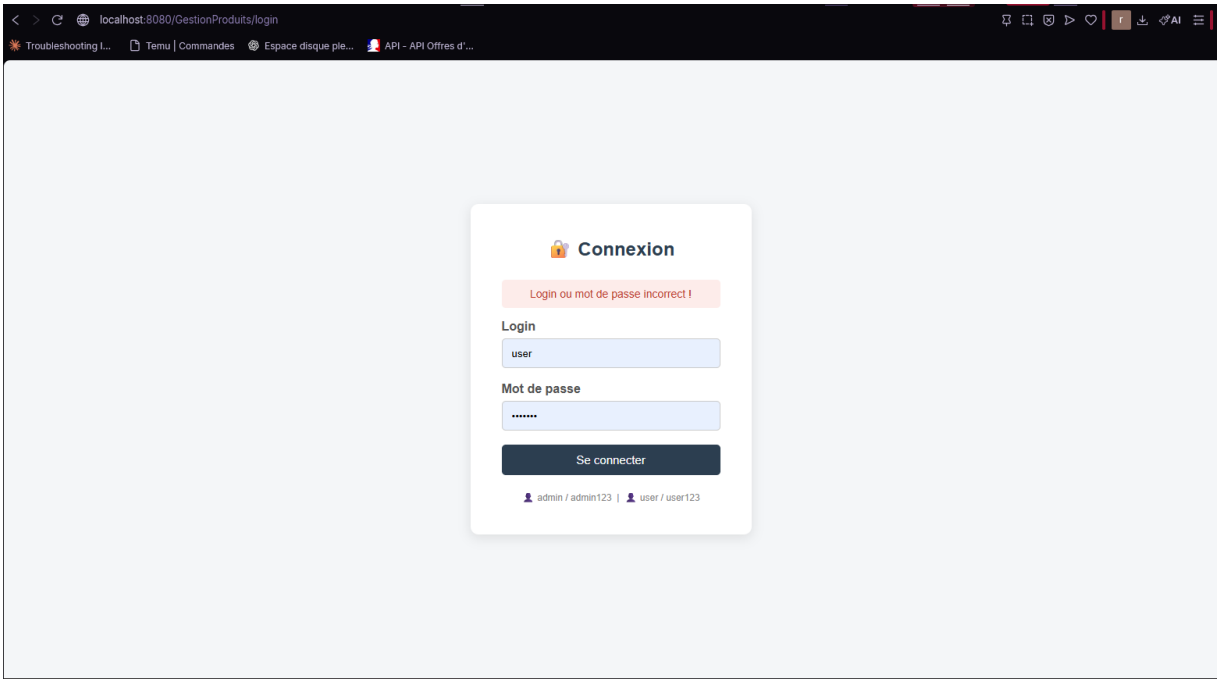

FIGURE 10.2 – Message d'erreur pour identifiants incorrects

10.2.2 Interface ADMIN

FIGURE 10.3 – Tableau des produits avec boutons Modifier/Supprimer

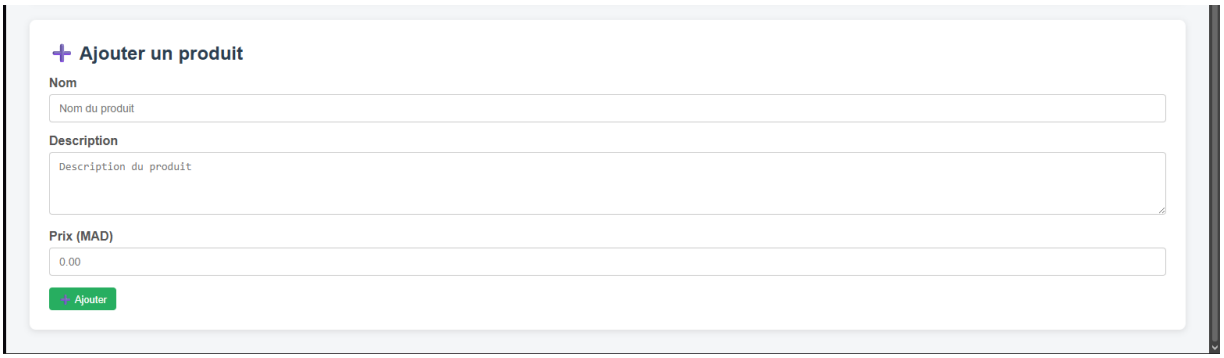

FIGURE 10.4 – Formulaire d'ajout d'un nouveau produit

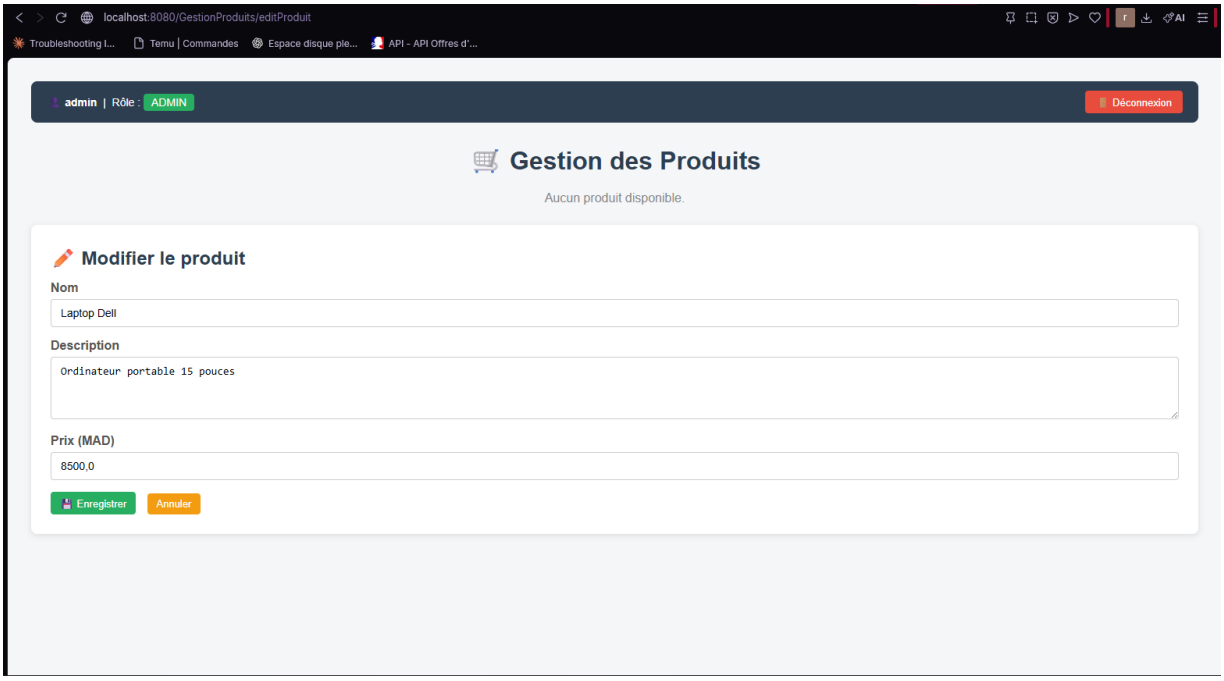

FIGURE 10.5 – Formulaire de modification d'un produit existant

10.2.3 Interface USER

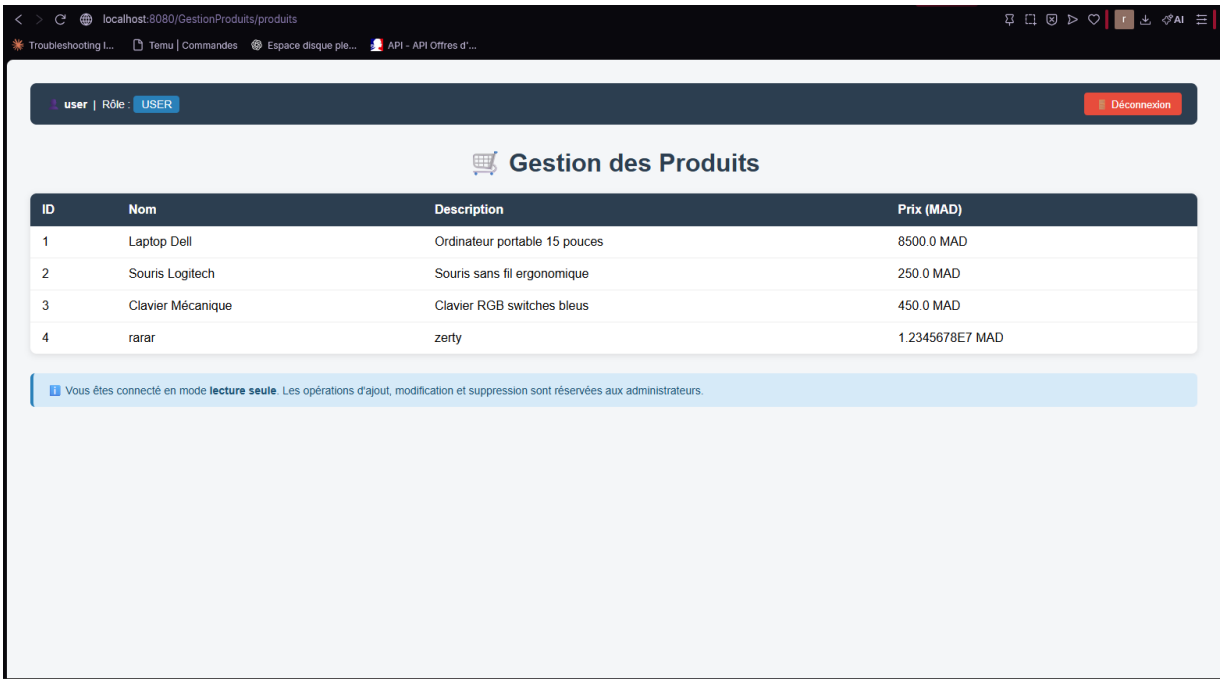

FIGURE 10.6 – Page d'accueil USER

10.2.4 Pages d'Erreur

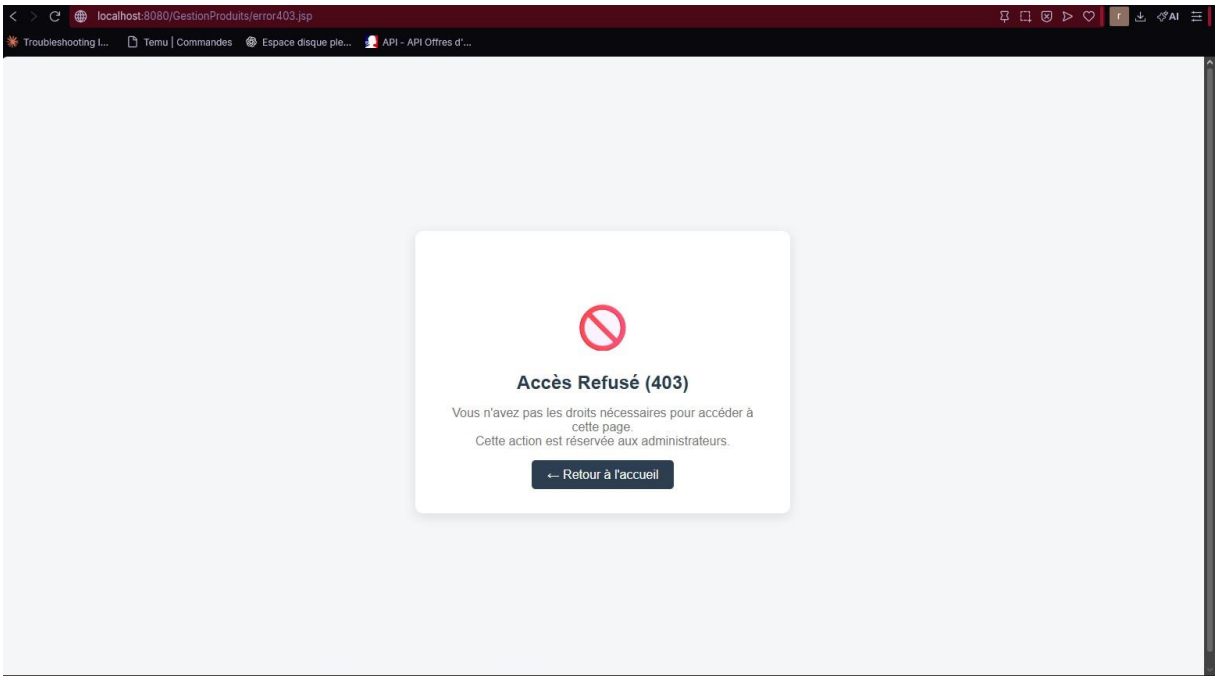

FIGURE 10.7 – Accès refusé pour un USER tentant une action ADMIN

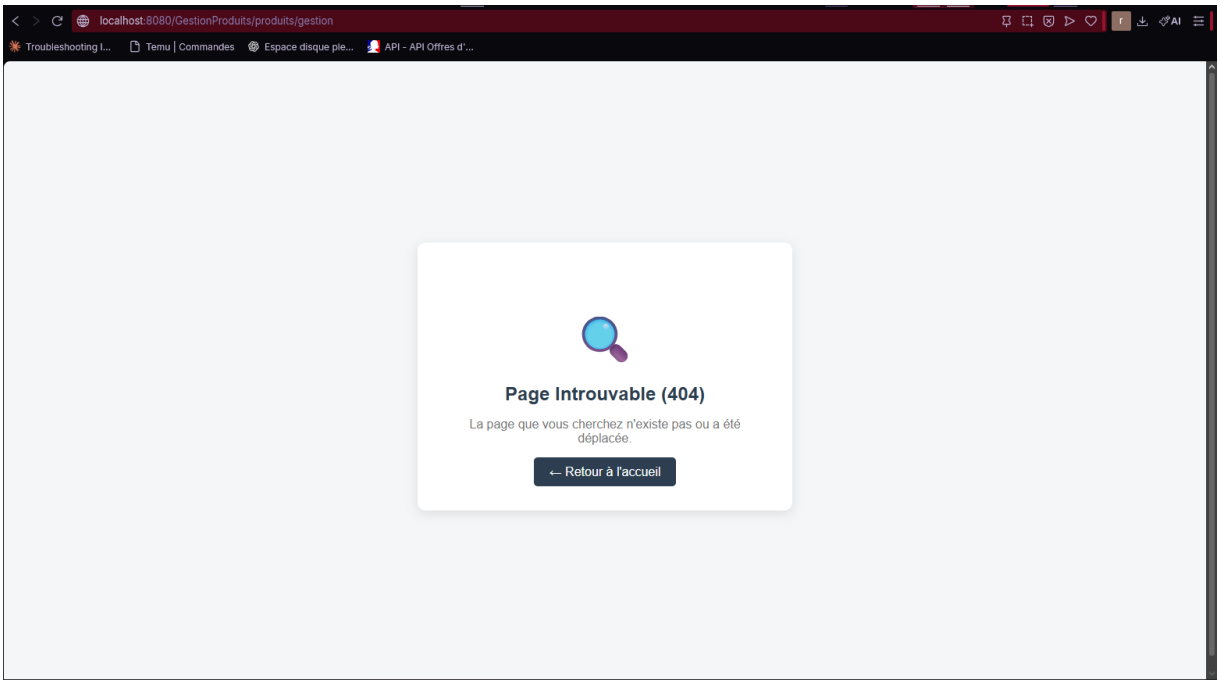

FIGURE 10.8 – Page introuvable

11. Problèmes Rencontrés et Solutions

11.1 Erreur 404 au démarrage

Problème

Après déploiement sur Tomcat 9, l'URL /GestionProduits/produits retournait une erreur 404.

Cause : Le projet utilisait les dépendances jakarta.* (compatibles Tomcat 10+) alors que le serveur installé était **Tomcat 9**.

Solution : Remplacer dans le pom.xml toutes les dépendances jakarta.* par leurs équivalents javax.*, et mettre à jour tous les imports dans les Servlets ainsi que l'URI JSTL dans les JSP.

TABLE 11.1 – Différences Tomcat 9 vs Tomcat 10

Tomcat 9 (javax.*)	Tomcat 10 (jakarta.*)
javax.servlet-api 4.0.1	jakarta.servlet-api 5.0.0
javax.servlet.jsp-api 2.3.3	jakarta.servlet.jsp-api 3.0.0
javax.servlet.jstl 1.2	jakarta.servlet.jsp.jstl 2.0.0
URI : java.sun.com/jsp/jstl/core	URI : jakarta.tags.core

11.2 Erreur de Doublons dans web.xml

Problème

Eclipse signalait des erreurs *"Duplicate unique value [AddProduitServlet]"*.

Cause : Lors de la mise à jour du web.xml, le nouveau contenu a été collé sans supprimer l'ancien, provoquant une double déclaration de chaque Servlet.

Solution : Sélectionner tout le contenu du web.xml (Ctrl+A) puis le remplacer intégralement par le nouveau contenu.

11.3 USER pouvait accéder aux actions ADMIN

Problème

Un utilisateur avec le rôle USER pouvait voir et utiliser les boutons Modifier et Supprimer.

Cause : Le RoleFilter bloquait bien les accès directs par URL, mais la vue index.jsp affichait les boutons sans vérification du rôle.

Solution : Ajouter des conditions JSTL <c:if> dans index.jsp pour masquer les boutons, le formulaire d'ajout et de modification selon le rôle de l'utilisateur connecté.

12. Conclusion

Ce travail pratique nous a permis de mettre en œuvre une application web Java EE complète en respectant l'architecture **MVC1** et l'organisation en **3 tiers**.

Les principaux acquis de ce TP sont :

— La maîtrise de l'architecture MVC1 avec **Servlet** comme contrôleur et **JSP/JSTL** comme vue.

— La mise en place du patron de conception **DAO** pour l'accès aux données.

— L'utilisation des **filtres Java EE** pour sécuriser l'application (authentification et autorisation).

— La gestion des **rôles utilisateurs** à la fois côté serveur (filtre) et côté vue (JSTL).

— La configuration d'un projet **Maven** pour une application web Java EE.

— La résolution de problèmes de compatibilité entre versions de Tomcat (`javax.*` vs `jakarta.*`).

En perspective, cette application pourrait être améliorée en intégrant une vraie base de données relationnelle via **JDBC** ou **JPA/Hibernate**, ou en migrant vers une architecture **MVC2** avec un Front Controller unique.

A. Comptes Utilisateurs de Test

TABLE A.1 – Comptes disponibles pour les tests

Login	Mot de passe	Rôle
admin	admin123	ADMIN (accès complet)
w	user123	USER (lecture seule)

B. URLs de l'Application

TABLE B.1 – URLs disponibles

URL	Méthode	Description
/login	GET/POST	Page de connexion
/logout	GET	Déconnexion
/produits	GET	Liste des produits
/addProduit	POST	Ajouter un produit
/editProduit	GET	Formulaire modification
/updateProduit	POST	Enregistrer modification
/deleteProduit	GET	Supprimer un produit